Context Quilt: System Architecture Document

Scalable Cognitive Memory Infrastructure for AI Applications

Version: 3.7
Date: November 30, 2025
Status: Implementation Ready - Comprehensive Technical Specifications

1. System Overview

1.1 Core Value Proposition: A Paradigm Shift in AI Memory

Context Quilt signifies a foundational advancement in the interaction paradigm between Artificial Intelligence and human users, transitioning from isolated, stateless exchanges to a continuous, cognitive engagement model. It operates as a sophisticated **Context Enrichment API**, empowering AI applications to retain a comprehensive memory of user interactions not merely within the confines of a single session, but across the entirety of the user lifecycle. This persistence spans days, weeks, and potentially years, transforming generic, one-size-fits-all AI responses into highly personalized, context-aware dialogues that foster trust, enhance efficiency, and ensure continuity. By transcending simple keyword matching to achieve a profound understanding of user preferences, history, and intent, Context Quilt enables applications to anticipate user needs proactively rather than simply reacting to explicit commands.
The system guarantees sub-20ms latency for context retrieval, ensuring that the user experience remains instantaneous and responsive—a non-negotiable requirement for modern, high-performance applications where even minor delays can degrade user satisfaction. Concurrently, Context Quilt employs a robust, multi-stage asynchronous learning pipeline to continuously refine its understanding of the user. This dual-path architecture facilitates deep, meaningful profile construction without compromising real-time performance, effectively resolving the historical trade-off between computational intelligence and system responsiveness. The architecture is inherently designed for horizontal scalability, capable of supporting millions of concurrent users while strictly adhering to latency service level agreements (SLAs).
Key Differentiator:
The contemporary landscape of AI memory solutions is predominantly characterized by two distinct approaches, each possessing inherent limitations when deployed in isolation:

- **Retrieval-Augmented Generation (RAG)** systems are highly effective at retrieving static documents and corporate knowledge bases to ground AI responses in factual

data. However, they fundamentally lack the capacity to track the dynamic, evolving state of an individual user. While a RAG system may understand company policy, it remains ignorant of the user's identity, role, or specific context. Consequently, RAG interactions are inherently impersonal, treating each query as a discrete event against a static corpus.

- **Vector Databases** serve as the foundational storage mechanism for semantic search, housing raw embeddings of text segments. While indispensable for similarity matching, they function as a low-level infrastructure component requiring significant engineering overhead to manage user identity, temporal relevance, and state effectively. They provide the storage medium—the "hard drive"—but lack the organizational logic—the "operating system"—necessary for coherent memory management, leaving developers to construct complex retrieval and context management logic from scratch.

Context Quilt bridges this critical functional gap by managing **Stateful User Memory**. It organizes user understanding into three distinct, interconnected layers:

- **Identity:** This layer encompasses immutable attributes such as the user's name, professional role, verified credentials, and unique identifiers. It forms the stable core of the user's digital twin, ensuring consistency and recognition across all interactions.
- **Preferences:** This layer captures explicit and implicit constraints, likes, and dislikes, such as dietary restrictions, preferred communication channels, and budget limitations. These parameters serve as the guiding principles for decision-making, enabling the AI to tailor recommendations and actions specifically to the individual.
- **Traits:** This layer consists of behavioral and stylistic characteristics inferred from ongoing interactions, such as technical proficiency, verbosity preference, and patience levels. These subtle cues allow the AI to adapt its *personality* and communication style to resonate with the user, fostering a more natural and engaging conversational experience.

This rich, structured context is seamlessly injected into Large Language Model (LLM) prompts via a transparent, developer-friendly **Template System**. This ensures that every interaction is informed by a comprehensive understanding of the user, eliminating the need for complex, ad-hoc prompt engineering by the developer. This abstraction layer simplifies the development process, allowing engineering teams to focus on creating superior application logic rather than managing intricate memory infrastructure.

1.2 The "Template API" Pivot: Empowering Developers

We have strategically transitioned away from the traditional LLM Gateway model, which functions as a heavy proxy for all LLM traffic. This pivot to a **Template-based Context Enrichment API** offers substantial advantages in terms of flexibility, reliability, and developer control. Instead of routing mission-critical LLM requests through our infrastructure—thereby creating a potential single point of failure, introducing a privacy bottleneck, and adding unavoidable network latency—developers invoke Context Quilt to *enrich* their prompts with context, subsequently calling their LLM provider directly. This decoupling ensures that the critical execution path of the application remains firmly under the developer's control.

Detailed Interaction Flow:

1. **Application Request:** The developer's application initiates the interaction by issuing a call to the /v1/enrich endpoint. The request payload includes a prompt template containing specific placeholders for dynamic content, such as User Profile: [[food_allergies]] or Strategy: [[guidance_strategy]]. This request is lightweight, containing only the essential metadata required for context retrieval.
2. **Context Resolution:** Upon receipt, Context Quilt processes the template. It parses the string to identify the requested variables (e.g., food_allergies). It then retrieves the corresponding values from its high-speed Redis cache or persistent PostgreSQL storage. This process involves resolving any complex logic, applying rigorous access control policies, and inserting default values defined in the schema if necessary. This step includes permission verification, privacy masking application, and data freshness validation.
3. **Enrichment Response:** Context Quilt returns the fully populated text string to the application. For example: User Profile: Peanuts, Shellfish. This response is purely text-based, lightweight, and formatted for immediate integration into any LLM prompt structure.
4. **LLM Interaction:** The application transmits this enriched text to its chosen LLM provider (e.g., OpenAI, Anthropic, or a local open-source model). This direct connection ensures that the application retains full sovereignty over its LLM configuration, cost management, model selection, and data privacy agreements with the provider. It also affords developers the flexibility to switch LLM providers without disrupting their underlying memory infrastructure.
5. **Asynchronous Learning:** Following the completion of the LLM interaction, the application transmits the full chat log (comprising both user input and assistant response) to the /v1/memory/update endpoint. This action triggers the asynchronous learning process, enabling Context Quilt to update its memory stores, refine user profiles, and extract novel insights without impacting the user's latency. This separation of concerns ensures that the computationally intensive "learning" phase never impedes the latency-sensitive "acting" phase.

2. Architecture: Dual-Path Processing

To reconcile the seemingly contradictory objectives of ultra-low latency retrieval and deep, computationally intensive learning, we employ a **Hot/Cold Architecture**. This design effectively decouples the read path from the write/process path, guaranteeing that user-facing operations are never blocked by backend processing. This architectural pattern is fundamental to maintaining system responsiveness at scale.

2.1 The Hot Path (Synchronous)

- **Goal:** The primary objective of the Hot Path is speed. We enforce a strict Service Level Agreement (SLA) of **< 20ms Response Time** (P99) for context retrieval to ensure that the integration of Context Quilt is imperceptible to the end-user. This stringent latency requirement is non-negotiable for real-time interactive applications.

- **Responsibility:** This path is exclusively dedicated to the **read-only retrieval** of pre-computed, cached context. It is deliberately designed to avoid complex logic, inference, or database writes that could introduce latency. It operates on the premise that the "heavy lifting" of data comprehension has already been executed by the Cold Path.
- **Infrastructure:** The Hot Path is powered by a high-performance **FastAPI** application server integrated with **Redis**, which serves as the Active Cache. This combination ensures minimal overhead and maximum throughput, capable of handling thousands of concurrent requests with negligible resource consumption.
- **Detailed Logic:**
    1. **Receive Template:** The API accepts the template string from the client application.
    2. **Parse Variables:** A highly optimized, regex-based parser scans the string for variable placeholders denoted by double brackets (e.g., [[variable_name]]). This parsing step is executed with minimal computational cost.
    3. **Fetch values from Redis:** The system executes a batched MGET lookup in Redis for all identified variables. This operation is extremely fast (sub-millisecond) as all relevant data keys are pre-calculated and stored in a flat, directly accessible format.
    4. **Access Control Check:** Prior to returning data, the system verifies the requesting App ID against the access policy for each variable (Raw, Pseudonymized, or Redacted). This step ensures that sensitive data is exposed only to authorized applications, enforcing a robust security posture.
    5. **Inject strings into Template:** The retrieved (and potentially masked) values are substituted into the template string. If a value is missing, predefined defaults from the schema are applied to prevent prompt breakage and ensure a consistent LLM experience.
    6. **Return to caller:** The final, enriched string is returned to the client application, ready for immediate transmission to the LLM.

Code Example: Hot Path Handler (FastAPI)

```
from fastapi import FastAPI, HTTPException, Request
from pydantic import BaseModel
import redis
import re
from typing import List, Dict, Optional

app = FastAPI()
# Redis connection pool configured for high performance and connection reuse
cache = redis.Redis(host='redis', port=6379, decode_responses=True)

class EnrichRequest(BaseModel):
    user_id: str
    template: str
```

```
    app_id: Optional[str] = None # Used for granular access control

@app.post("/v1/enrich")
async def enrich_prompt(request: EnrichRequest):
    # 1. Parse Variables
    # Regex efficiently identifies all instances of [[variable_name]] or [[variable|default]]
    matches = re.findall(r'\[\[(.*?)(?:\|(.*?))?\]\]', request.template)

    response_text = request.template

    if not matches:
        return {"enriched_prompt": request.template}

    # 2. Batch Fetch from Redis
    # Keys are stored using a consistent namespace: "var:{user_id}:{variable_name}"
    keys = [f"var:{request.user_id}:{var}" for var, default in matches]
    values = cache.mget(keys)

    # 3. Inject Values with Access Control Logic (Simplified)
    for (var, default), value in zip(matches, values):
        # Placeholder for access policy check logic
        # access_level = get_access_level(request.app_id, var)
        # value = apply_masking(value, access_level)

        # Use cached value if found, otherwise default, otherwise empty string
        final_val = value if value is not None else (default if default else "")

        # Replace in template
        placeholder = f"[[{var}|{default}]]" if default else f"[[{var}]]"
        response_text = response_text.replace(placeholder, final_val)

    return {"enriched_prompt": response_text}
```

2.2 The Cold Path (Asynchronous)

- **Goal:** The Cold Path is engineered for **Deep Learning & Extraction**. Because this process occurs after the user interaction is complete, there is no stringent latency requirement (zero user-facing impact). This affords the system the capability to utilize more computationally intensive models and logic to extract high-quality insights that would be impossible to generate in real-time.
- **Responsibility:** This path constitutes the "brain" of the system: extracting hard facts from conversation logs, inferring subtle user traits, updating communication profiles, and summarizing episodic memories. It is the locus of intelligence where the user model

is continuously refined based on new data.

- **Infrastructure:** The Cold Path is orchestrated by a **Python Worker** (utilizing task queues such as Celery or Temporal) that manages job execution. It leverages **Mistral 7B** (running as a Local LLM via Ollama) for intelligence and **PostgreSQL** as the "Cold Storage" vault. Utilizing a local LLM ensures strict data privacy by keeping inference within the customer's infrastructure and eliminates per-token costs associated with cloud-based LLMs.
- **Detailed Logic:**
    1. **Receive Chat Log:** The worker ingests a new chat log event from the Redis Stream (memory.update). This event contains the full context of the interaction, including metadata such as timestamps and user IDs.
    2. **Run "The Detective":** Mistral 7B analyzes the text to extract structured facts (e.g., "I live in Chicago") based on the defined schema and extraction rules. It employs Chain-of-Thought prompting to ensure accuracy and handle complex extraction logic.
    3. **Run "The Archivist":** Mistral 7B generates a concise, semantic summary of the interaction for long-term episodic recall. This summary strips away PII and focuses on intent, outcome, and key entities, optimizing it for vector search.
    4. **Run "The Strategist":** Mistral 7B synthesizes a high-level behavioral guidance strategy for future interactions (e.g., "User prefers direct answers. Avoid suggesting Shinjuku."). This strategy is stored as a variable and injected into future prompts to guide the LLM's behavior.
    5. **Run "The Psychologist":** Mistral 7B analyzes the user's communication style (tone, verbosity, technical level) to construct a psychometric profile using Few-Shot prompting. This profile enables the AI to adapt its communication style to align with the user's preferences.
    6. **Normalization & Validation:** Python code validates the extracted JSON against the schema, correcting any formatting errors and reclassifying "inferred" variables to the correct buckets. This step ensures data integrity and prevents "hallucinated" variables from polluting the database.
    7. **Update PostgreSQL:** All extracted and inferred data is persisted to the PostgreSQL database (The Vault), ensuring durability and consistency. This database serves as the definitive system of record.
    8. **Invalidate/Update Redis:** The updated context is pushed to the Redis Active Cache, making it immediately available for the next "Hot Path" request. This guarantees that the user's profile is always current for subsequent interactions.

3. Data Architecture: The "Hydration" Model

We have streamlined the database stack to minimize operational complexity while maximizing performance. We utilize **PostgreSQL** as the central "Vault" for all persistent data and **Redis** as the "Active Context" layer for speed. This "Hydration" model ensures data is always where it needs to be, precisely when it is needed, optimizing for both storage efficiency and access

speed.

3.1 Storage Layers

Tier	Technology	Role	Data Types and Usage
Cold Storage	**PostgreSQL**	**The Vault (Source of Truth)**	This is the permanent repository of all user data. It acts as the authoritative store for the system. It stores: • **Identity:** Core attributes like Name, Email, and Critical Allergies. These are high-value facts that rarely change. • **Mutable Profile:** Attributes that change over time, such as Current Address or Job Title. The system tracks the current state of these variables. • **Episodic:** Summarized events and interaction histories, stored using pgvector for semantic search capabilities. This allows the system to "remember" past conversations based on meaning. • **Access Policies:** Rules defining which apps can see which variables, ensuring

			granular data governance.
Hot Storage	**Redis**	**Active Context (Cache)**	This layer holds the data currently needed for active sessions. It acts as a high-speed buffer for the application. It stores: • **Session State:** The most recent conversation turns for immediate context, allowing the LLM to maintain coherence within a chat. • **Hydrated Profile:** A flattened JSON object containing all active variables for a user, ready for instant injection into prompts. This avoids complex database queries during the request. • **Locks & Rate Limits:** Operational data to manage concurrency and API usage limits, ensuring system stability.

3.2 The Pre-fetch ("Hydration") Workflow

The "Hydration" model ensures that data is moved from Cold to Hot storage *before* it is critically needed, eliminating database latency from the user's path. This proactive approach is key to achieving our low-latency goals.

1. **Trigger:** The application explicitly calls the /v1/prewarm endpoint when a user session starts (e.g., upon login), or the system automatically triggers this on the first /enrich call if the cache is detected as cold. This signal initiates the hydration process.
2. **Action:** The system executes a "heavy" query against PostgreSQL to fetch the

complete User Profile, relevant Shared Context, and a selection of Recent Episodes. This query is optimized for throughput but happens in the background, decoupled from the user's request flow.

3. **Cache:** The retrieved data bundle is serialized into a high-speed format (e.g., JSON or MessagePack) and stored in Redis under a key like active_context:user_123. A Time-To-Live (TTL) (e.g., 30 minutes) is applied to ensure freshness and automatically manage memory usage for inactive users, preventing the cache from growing indefinitely.
4. **Usage:** All subsequent /enrich calls for that user act as "cache hits," reading exclusively from Redis. This guarantees consistent, sub-millisecond latency for the duration of the user's session, regardless of the complexity or size of their long-term history.

Example: JSON Structure for active_context (in Redis)

```
{
  "user_id": "user_789",
  "variables": {
    "user_name": "Sarah",
    "food_allergies": "peanuts, shellfish",
    "communication_style": "concise, direct",
    "guidance_strategy": "Prioritize Apple products. Avoid Windows recommendations."
  },
  "session_history": [
    {"role": "user", "content": "I need headphones"},
    {"role": "assistant", "content": "I recommend AirPods Max"}
  ],
  "metadata": {
    "last_updated": "2025-11-29T10:00:00Z",
    "ttl_expires": "2025-11-29T10:30:00Z",
    "originating_app": "app_medical_01"
  }
}
```

4. Variable Discovery & Definition

We support a **Declarative Memory Model** that places developers in control. Developers define *what* information they want to track (the schema), and the Context Quilt system handles the complex logic of *how* to extract that information from unstructured conversation. This dramatically lowers the barrier to entry for creating smart, memory-aware agents, abstracting away the complexity of prompt engineering and regex.

4.1 The "Setup-Time Architect"

Defining extraction rules for LLMs can be tricky (e.g., writing regex or designing prompt instructions). To solve this, we introduce the **"Setup-Time Architect."** This feature uses a

powerful Frontier Model (like Gemini 1.5 Pro or GPT-4) during the configuration phase to generate robust extraction rules automatically. By leveraging the superior reasoning capabilities of these large models offline, we enable high-quality extraction at runtime with smaller, cheaper models.

- **Developer Input:** The developer simply states their intent in plain English: "I want to track noise_tolerance."
- **AI Architect Output:** The Frontier Model generates a comprehensive set of rules and logic: *"Map phrases like 'too busy', 'loud', 'noisy' -> Low. Map 'lively', 'party', 'downtown' -> High."* It handles synonyms, slang, and edge cases that a developer might overlook.
- **Result:** These generated rules are saved to the memory_schema.yaml file, providing high-quality extraction logic without manual engineering effort. This file serves as the blueprint for the runtime worker.

4.2 The Variable Lifecycle

Variables in Context Quilt follow a defined lifecycle, allowing for organic growth of the user profile schema based on actual data. This evolutionary approach ensures the schema remains relevant to real-world usage.

Stage	Status	Behavior and Cost Implications
1. Candidate	**Discovered**	The system (via Mistral 7B in Discovery Mode) has detected a recurring pattern in user conversations (e.g., users frequently mentioning "Shoe Size"). This creates a "Candidate" suggestion in the Developer Dashboard. **Zero Runtime Cost** for the developer, as this is a byproduct of the standard extraction process.
2. Registered	**Dormant**	The developer has reviewed and approved the variable. It is added to the Schema definition but is not yet referenced in any active prompt templates. The system tracks it in the database, but does not load it into hot cache. **Storage Cost Only.** This allows developers to collect data without incurring the

		memory overhead of caching it until needed.
3. Active	**Live**	The developer has added the variable tag (e.g., [[shoe_size]]) to a live template. The system now actively pre-fetches, caches, and injects this data for every request. **RAM Cost.** This state indicates the variable is critical for real-time operations.

Example memory_schema.yaml

```yaml
# Generated by Setup-Time Architect
version: "1.0"
application: "travel_bot"

variables:
  - name: "noise_tolerance"
    type: "enum"
    options: ["low", "medium", "high"]
    description: "User's sensitivity to noise levels in accommodations."
    # Rules generated by Architect
    extraction_rules:
      - pattern: ["quiet", "peaceful", "silent", "calm"]
        map_to: "low"
      - pattern: ["party", "lively", "nightlife", "busy"]
        map_to: "high"

  - name: "budget_tier"
    type: "enum"
    options: ["budget", "standard", "luxury"]
    description: "Inferred spending power based on requests."
    extraction_rules:
      - pattern: ["cheap", "low cost", "hostel", "deal"]
        map_to: "budget"
      - pattern: ["5 star", "premium", "best available", "suite"]
        map_to: "luxury"
```

5. The "Four-Prompt" Cold Path Pipeline

Our Async Worker utilizes **Mistral 7B** (running locally on GPU infrastructure) to execute a

sophisticated memory consolidation pipeline. This pipeline is broken down into four specialized prompts, each designed to perform a specific cognitive task with high accuracy. This modular approach prevents the model from getting "confused" by trying to do too much at once and allows for targeted optimization of each task.

5.1 Prompt 1: The Archivist (Summarization)

- **Task:** The Archivist's role is to compress the raw, verbose chat log into a dense, semantic paragraph. It strips away "chitchat" and focuses on key events, decisions, and outcomes.
- **Output:** The summary is vectorized and stored in the episodic_memory table in PostgreSQL (using pgvector).
- **Goal:** This enables future semantic search queries, allowing the system to answer questions like "Why did we pick Ueno?" by retrieving the relevant historical context. By converting dialogue into narrative summary, we significantly improve retrieval relevance.

Archivist Prompt Template:

```
<s>[INST] You are the Archivist. Compress this chat into a dense summary for vector search.
Remove all chitchat. Focus strictly on Intent, Constraints, and Outcomes.
Write in the past tense (e.g. "User asked about...").

[CHAT LOG]
{{ transcript }}
[/INST]
```

5.2 Prompt 2: The Detective (Extraction)

- **Task:** The Detective is responsible for structured data extraction. It scans the conversation for values corresponding to **Known Variables** (defined in the schema) and identifies potential **Candidate Variables** (new patterns).
- **Technique:** It uses **Chain of Thought (CoT)** prompting. The model is instructed to *"Think first, then extract,"* writing out its reasoning before generating the final JSON. This significantly reduces hallucinations and improves accuracy by forcing the model to justify its extraction decisions.
- **Output:** A structured JSON object containing known_variables and candidate_variables.
- **Safety:** A Python normalization layer post-processes this output, ensuring that any "candidates" that match known schema keys are correctly moved to the "known" bucket. This handles cases where the model might misclassify known data as new discoveries.

Detective Prompt Template:

```
<s>[INST] You are the Data Detective. Extract structured user data.

[KNOWN VARIABLES SCHEMA]
{{ schema_list }}
[RULES]
```

{{ rule_list }}

[INSTRUCTIONS]
1. First, write a "THOUGHT PROCESS". Go through the transcript. Identify evidence for each variable.
2. Then, output the "FINAL JSON" block.
  - Put mapped values into "known_variables".
  - Put new discoveries in "candidate_variables".

[CHAT LOG]
{{ transcript }}
[/INST]

5.3 Prompt 3: The Psychologist (Profiling)

- **Task:** The Psychologist analyzes the **User's** communication style, independent of the content. It evaluates dimensions such as Tone (Formal/Casual), Verbosity (Concise/Detailed), and Technical Level (Novice/Expert).
- **Technique:** It uses **Few-Shot Prompting**, providing the model with concrete examples of input/output pairs to enforce the correct analysis format. This is crucial for getting consistent, structured output from smaller models.
- **Input:** The prompt is fed only the user's messages (stripped of Assistant text) to prevent the model from confusing the bot's polite tone with the user's actual style.
- **Output:** A JSON Profile object (e.g., verbosity: "concise", technical_level: "expert").

Psychologist Prompt Template:
<s>[INST] You are a data extraction engine. You do NOT converse. You ONLY output JSON. Analyze the conversation and extract a communication profile.

[EXAMPLE INPUT]
User: "no thanks"
[EXAMPLE OUTPUT]
REASONING
User is concise and blunt.
JSON
{ "dimensions": { "verbosity": { "value": "Ultra-Concise", ... } } }

[REAL INPUT DATA]
{{ user_only_transcript }}

[DIMENSIONS]
1. Verbosity (Ultra-Concise, Concise, Balanced...)
2. Tone (Formal, Casual, Blunt...)
... (full list of 10 dimensions)

Output ONLY Reasoning and JSON.
[/INST]

Token Optimization Strategy:
To mitigate the potential token cost associated with extensive profiling (approximately 450-500 tokens per full JSON profile), we implement specific optimization strategies.

- **Progressive Profiling:** The full profile computation is deferred until a sufficient volume of interaction data (e.g., exceeding 5 messages) has been accumulated.
- **Summary Format:** For standard interactions, a distilled summary (e.g., "User prefers concise, direct answers") is utilized. This approach reduces token consumption by approximately 85% (to roughly 60 tokens) while retaining an estimated 90% of the personalization value.
- **Caching:** The profile is computed once per session or triggered only upon significant updates, rather than being recalculated with every turn.

5.4 Prompt 4: The Strategist (Synthesis)

- **Task:** The Strategist converts the raw data and profile into actionable **Behavioral Guidance**. It synthesizes a strategy string that tells the LLM *how* to interact with this specific user.
- **Technique:** It uses **Negative Constraint** enforcement. The prompt explicitly instructs the model to list "Avoid" items separately, preventing negative preferences (e.g., "I hate Shinjuku") from accidentally becoming recommendations. This is a critical safety mechanism.
- **Output:** A concise strategy string (e.g., *"Guidance: User values quiet. Priority: Ueno. Avoid: Shinjuku."*).
- **Usage:** This string is injected into the system prompt via the [[guidance_strategy]] variable, guiding the LLM's behavior without requiring complex prompt engineering from the developer. It essentially "programs" the LLM with the user's preferences.

Strategist Prompt Template:
<s>[INST] You are the Strategy Engine. Synthesize raw user data into a Strategy.

[INPUT DATA]
{{ extracted_facts }}

[CRITICAL INSTRUCTIONS]
1. Analyze "DISLIKES/REJECTIONS" first. These must be listed in an "Avoid" section.
2. Synthesize the strategy into this exact format:
   "Guidance: [Summary]. - Priority: [Recommendations]. - Avoid: [Rejections]. - Tone: [Style]."

Constraint: Keep it under 60 words.
[/INST]

6. Implementation & Cost Model

6.1 Infrastructure

- **Compute:** We utilize Google Cloud **g2-standard-4** instances, which are equipped with the **NVIDIA L4 GPU**. This provides an optimal balance of price and performance for inference workloads, offering enough VRAM to run Mistral 7B efficiently.
- **Strategy:** To minimize costs, we use **Spot Instances** for our Async Workers. Since memory consolidation is not time-critical (it can be delayed by minutes without impact), we can tolerate interruptions, achieving **60-90% savings** on compute costs. The worker architecture handles interruptions gracefully by requeuing tasks.
- **Orchestration:** Deployment is managed via Docker Compose for single-node setups or Kubernetes (K8s) for scaled production environments, providing flexibility for different customer needs.

6.2 Cost Estimate (1,000 Conversations)

- **Workload:** Processing 1,000 conversations requires approximately 4,000 inference calls to Mistral 7B (4 prompts per conversation).
- **Runtime:** Based on benchmarks, this workload consumes roughly **35 minutes of GPU time**.
- **Cost:** At current Spot Instance pricing (~$0.23/hr for L4 Spot), the total cost for processing 1,000 conversations is **<$0.50**. This confirms the economic viability of the architecture even at scale, making it cost-effective to offer deep memory capabilities.

6.3 Deployment Deliverable

We ship a complete, self-contained Docker solution that is easy to deploy and manage. This approach simplifies the customer's operational burden.

1. **api**: The FastAPI Gateway handling the Hot Path and API requests.
2. **worker**: The Python Celery Worker managing the Cold Path processing.
3. **ollama**: The Local Inference Server running the Mistral 7B model.
4. **redis**: The in-memory store for Caching and Task Queues.
5. **postgres**: The persistent database for Storage and Vector Search.

docker-compose.yml Snippet:

```
services:
  ollama:
    image: ollama/ollama:latest
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
```

```
        count: 1
        capabilities: [gpu]
command:
  - |
    ollama serve &
    sleep 5
    echo "Pulling Mistral 7B..."
    ollama pull mistral
    wait
```